Makine Öğrenmesi

SD413

Yapay Sinir Ağları

- Yapay sinir ağlarında, nöron adı verilen birçok basit birim, daha büyük yapılar halinde birbirine bağlanır.
- Ağın sınıflandırma davranışı, nöronları birbirine bağlayan bağlantıların ağırlıkları tarafından belirlenir.
- Makine öğrenmesinin görevi, iyi sınıflandırma davranışıyla sonuçlanan ağırlıkları bulabilen algoritmalar sağlamaktır.
- Bu arama, genellikle bir sinir ağının eğitimi olarak adlandırılan bir süreçle gerçekleştirilir.

Yapay Sinir Ağları

- Sinir ağları teorisi ve pratiği, yalnızca bu dersle tüketilemeyecek kadar geniştir. Bu nedenle, konunun temsilcisi olarak yalnızca çok katmanlı algılayıcılar (multilayer perceptrons, MLP) ele alınacaktır.
- Çok katmanlı algılayıcılar dışında radial basis function networks, convolutional neural networks, recurrent neural networks, vb.
- MLP artık çok popüler olan **derin öğrenme** paradigması ile ilişkileri sayesinde oldukça güncel hale gelmiştir.

Çok Katmanlı Algılayıcılar

- Şimdilik tüm özniteliklerin sürekli olduğunu ve [–1, 1] aralığına normalize edildiğini varsayarak işimizi basitleştirelim. Normalleştirme şart değildir, ancak kesinlikle faydalıdır.

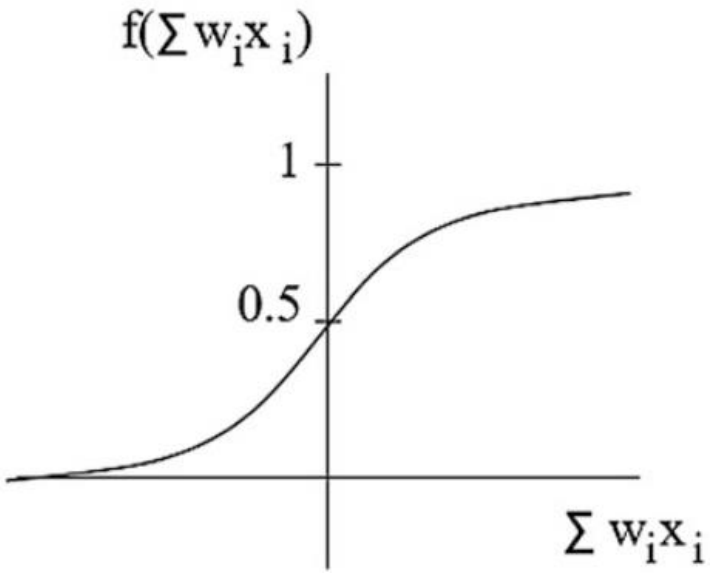
Çok Katmanlı Algılayıcılar

Nöronlar ve Sigmoid Fonksiyon: Çok katmanlı bir algılayıcının temel birimi olan bir nöronun işlevi basittir: girdiye gelen sinyallerin ağırlıklı bir toplamı, nöronun çıktısını sağlayan bir transfer işlevine (aktivasyon işlevi olarak da bilinir) gönderilir. Birkaç farklı aktarım işlevi kullanılabilir. Başlangıç olarak, girdilerin ağırlıklı toplamının olduğu aşağıdaki formülle tanımlanan sigmoid'e göz atalım:

$$f(\Sigma) = \frac{1}{1 + e^{-\Sigma}}$$

Çok Katmanlı Algılayıcılar

- Aşağıdaki şekil sigmoidin neye benzediğini göstermektedir. f(Σ) fonksiyonunun Σ ifadesinin artan değeriyle monoton bir şekilde büyüdüğüne dikkat edin. Dikey eksen f(0)=0.5'te kesişir. Genellikle ağdaki tüm nöronlar için aynı aktivasyon fonksiyonu kullanılır.



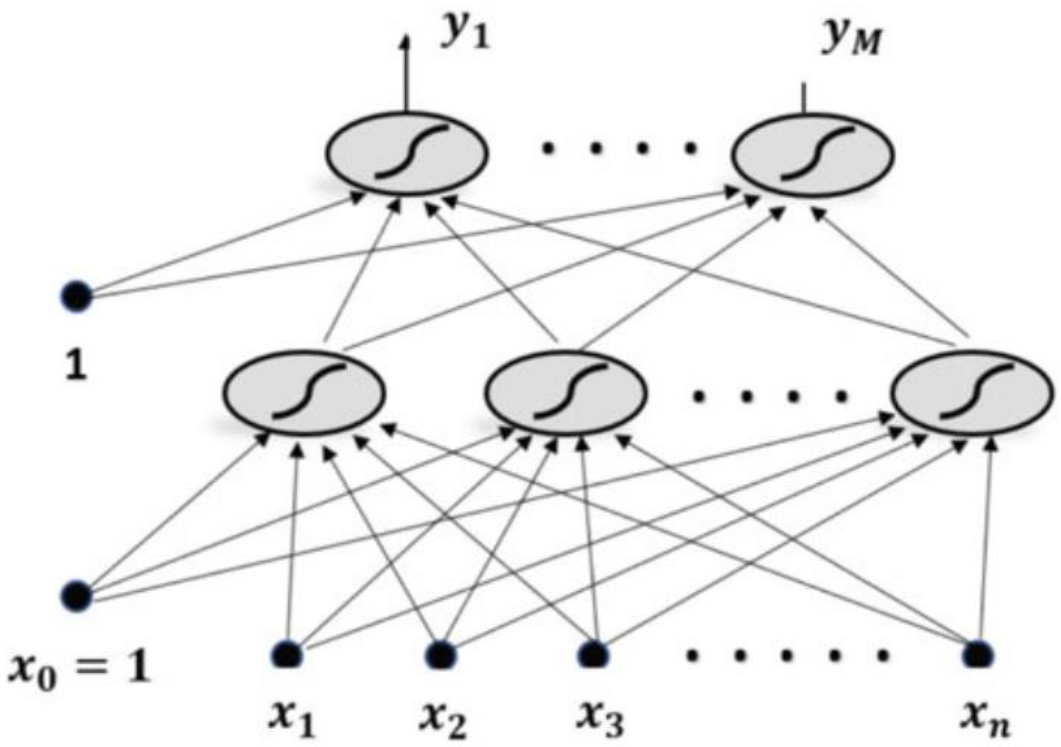
Sigmoid’in Birinci Türevi

- Sigmoid'in sinir ağları alanında bu kadar popüler olmasının belki de temel nedeni, birinci türevinin "orijinal fonksiyonun bir fonksiyonu" şeklinde ifade edilebilmesidir.

$$f'(\Sigma) = f(\Sigma)[1 - f(\Sigma)]$$

Çok Katmanlı Algılayıcılar

- Aşağıdaki sinir ağı, çok katmanlı algılayıcı olarak bilinir. Ovallerle temsil edilen nöronlar iki katman halinde düzenlenmiştir: çıkış katmanı ve gizli katman.



Çok Katmanlı Algılayıcılar

- Kolaylık sağlamak için, şimdi yalnızca tek bir gizli katmana sahip ağları ele alacağız.
- Nöronların eğitilebilir yanlılığını sağlamak için sıfırıncı ağırlıkları da kullandıklarına dikkat edin.
- Aynı katmandaki nöronlar arasında iletişim yokken bitişik katmanlar tamamen birbirine bağlıdır.
- Daha da önemlisi, her nörondan nörona bağlantı bir ağırlıkla ilişkilendirilir. j-inci gizli nörondan i-inci çıkış nöronuna olan bağlantının ağırlığı $w_{ji}^{(1)}$ olarak ve k-inci öznitelikten j-inci gizli nörona giden bağlantının ağırlığı $w_{kj}^{(2)}$ olarak gösterilir.
- İlk indeksin bağlantının nerede başladığını ve ikinci indeksin bize nerede bittiğini gösterdiğini unutmayın. Üst simge, (1) veya (2), katmanı belirtir.

İleri Yayılım (Forward Propagation)

- Ağın girdisinin bir örnekle sunulduğunu varsayalım, $x = (x_1, \ldots, x_n)$. Nitelik değerleri, her bir x_k'yi karşılık gelen bağlantılarla ilişkili ağırlıklarla çarpan nöral bağlantılar boyunca iletilir.

- Örneğin, j'inci gizli nöron ağırlıklı toplamı, $\sum_k w_{kj}^{(2)} x_k$ 'yı girdi olarak alır ve bu toplam daha sonra sigmoid aktivasyon fonksiyonuna, f $(\sum_k w_{kj}^{(2)} x_k)$ tabi tutulur.

- i. çıkış nöronu daha sonra gizli nöronlardan gelen değerlerin ağırlıklı toplamını alır ve toplamı yine sigmoid aktivasyon fonksiyonuna tabi tutar. Ağın i. çıkışı bu şekilde elde edilir. Öznitelik değerlerinin ağın girişinden çıkışına bu şekilde yayılma işlemine ileri yayılım denir.

İleri Yayılım (Forward Propagation)

- Sınıflandırma için çok katmanlı algılayıcıları kullanırken, her sınıfa bir çıkış nöronu atarız ve i. çıkış nöronu tarafından döndürülen değer, i. sınıfı destekleyen kanıt olarak yorumlanır. Örneğin, üç çıkış nöronunda elde edilen değerler y = (0.2, 0.6, 0.1) ise, sınıflandırıcı verilen örneği ikinci sınıfla etiketleyecektir çünkü 0.6, 0.2 ve 0.1'den büyüktür.

- Aslında, iki katmanlı MLP aşağıdaki formülle temsil edilir; burada f, sigmoid transfer fonksiyonunu, $w_{kj}^{(2)}$ ve $w_{ji}^{(1)}$ sırasıyla gizli ve çıkış katmanlarına giden bağlantıların ağırlıklarını, x_k ise ağa sunulan örneğin öznitelik değerlerini göstermektedir.

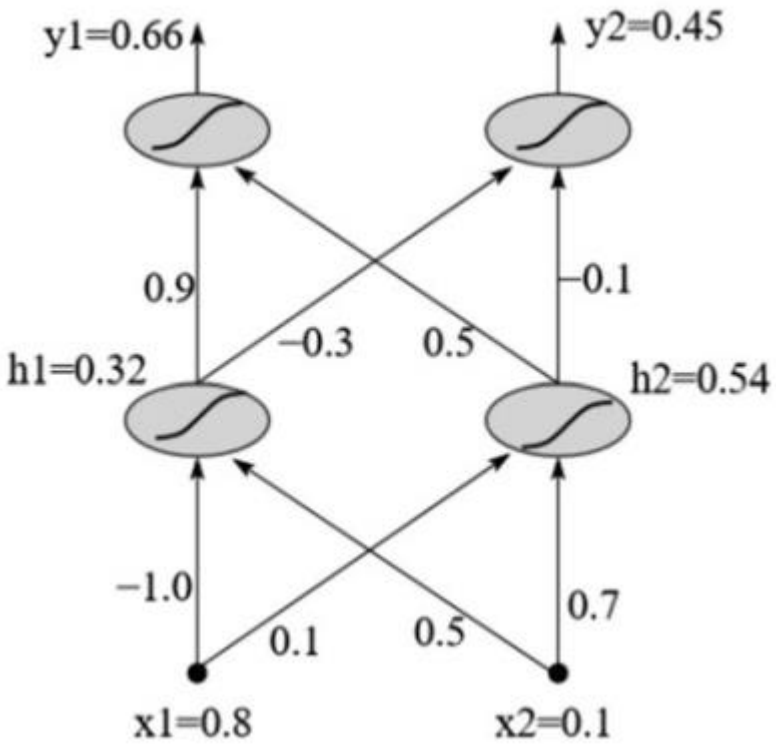
$$y_i = f\left(\sum_j w_{ji}^{(1)} f\left(\sum_k w_{kj}^{(2)} x_k\right)\right)$$

Çok Katmanlı Algılayıcılar

MLP Evrensel Bir Sınıflandırıcıdır

- Matematikçiler, doğru ağırlık seçimi ve doğru gizli katman boyutu ile herhangi bir gerçek fonksiyona keyfi isabetle yaklaşılabileceğini kanıtlamışlardır.
- Bu evrensellik teoreminin sonucu, çok katmanlı algılayıcının prensipte hemen hemen her türlü sınıflandırma görevini ele almak için kullanılabilmesidir.
- Bununla birlikte, teoremin bize söylemediği şey, kaç tane gizli nörona ihtiyaç duyulduğu ve bireysel ağırlıkların değerlerinin ne olması gerektiğidir. Başka bir deyişle, çözümün var olduğunu biliyoruz ama onu bulacağımızın garantisi yok.

Örnek



Inputs of hidden-layer neurons:

$$z_1^{(2)} = 0.8 \times (-1.0) + 0.1 \times 0.5 = -0.75$$

$$z_2^{(2)} = 0.8 \times 0.1 + 0.1 \times 0.7 = 0.15$$

Outputs of hidden-layer neurons:

$$h_1 = f(z_1^{(2)}) = \frac{1}{1+e^{-(-0.75)}} = 0.32$$

$$h_2 = f(z_2^{(2)}) = \frac{1}{1+e^{-0.15}} = 0.54$$

Inputs of output-layer neurons:

$$z_1^{(1)} = 0.32 \times 0.9 + 0.54 \times 0.5 = 0.56$$

$$z_2^{(1)} = 0.32 \times (-0.3) + 0.54 \times (-0.1) = -0.15$$

Outputs of output-layer neurons:

$$y_1 = f(z_1^{(1)}) = \frac{1}{1+e^{-0.56}} = 0.66$$

$$y_2 = f(z_2^{(1)}) = \frac{1}{1+e^{-(-0.15)}} = 0.45$$

Sinir Ağı İçin Hata

- MLP'nin eğitim tekniğini tanıtmadan önce, sınıflandırma kararlarının doğruluğunu nasıl ölçeceğimize daha yakından bakmamız gerekiyor.
- Çok katmanlı algılayıcıya bilinen sınıfı c(x) olan x örneğini sunarak başlayalım. İleriye doğru yayılma, x'in h(x) ile etiketlenmesiyle sonuçlanır. $h(x) \neq c(x)$ ise sınıflandırma yanlıştır.
- Bu, diğer örneklerde de olabilir ve mühendis bunun ne sıklıkta olduğunu yani hata oranını bilmek ister. Hata oranı, hata sayısının sınıflandırılan örnek sayısına bölünmesiyle hesaplanır. Örneğin, sınıflandırıcı 200 örnekten 30'unu yanlış sınıflandırırsa, hata oranı 30/200 = 0,15 olur.
- Bununla birlikte, hata oranı, sınıflandırma davranışının kaba bir resmini çizer. Yansıtamadığı şey, sigmoid'in her bir hatanın boyutunu ölçme yeteneğidir.

Sinir Ağı İçin Hata

- Toplam hatayı bulmak için çeşitli ölçütler kullanılır. Bunlardan biri ortalama karesel hatadır (Mean Squared Error).

$$MSE = \frac{1}{m}\sum_{i=1}^{m}(t_i - y_i)^2$$

- Ağın MSE'sini elde etmek için, her çıkış nöronunun çıkışı ile hedef vektörün karşılık gelen elemanı arasındaki farkı belirlememiz gerekir.

Sinir Ağı İçin Hata

Örnek

- Çıkış vektörü y(x) = (0.5, 0.2, 0.9) ve hedef vektör t(x) = (0, 1, 0) ise, ortalama karesel hatalar aşağıdaki değerlere sahiptir:

$$MSE = 1/3[(0 - 0.5)^2 + (1 - 0.2)^2 + (0 - 0.9)^2)] = 0.57$$

Hatanın Geri Yayılımı (Back Propagation)

- Çok katmanlı algılayıcılarda, ağın davranışı eğitilebilir ağırlıklar $w_{ji}^{(1)}$ ve $w_{kj}^{(2)}$ tarafından belirlenir.
- Makine öğrenmesinin görevi, sınıflandırma performansını optimize eden ağırlıkları bulmaktır. Bu eğitim ile elde edilir.

Hatanın Geri Yayılımı (Back Propagation)

Genel Senaryo

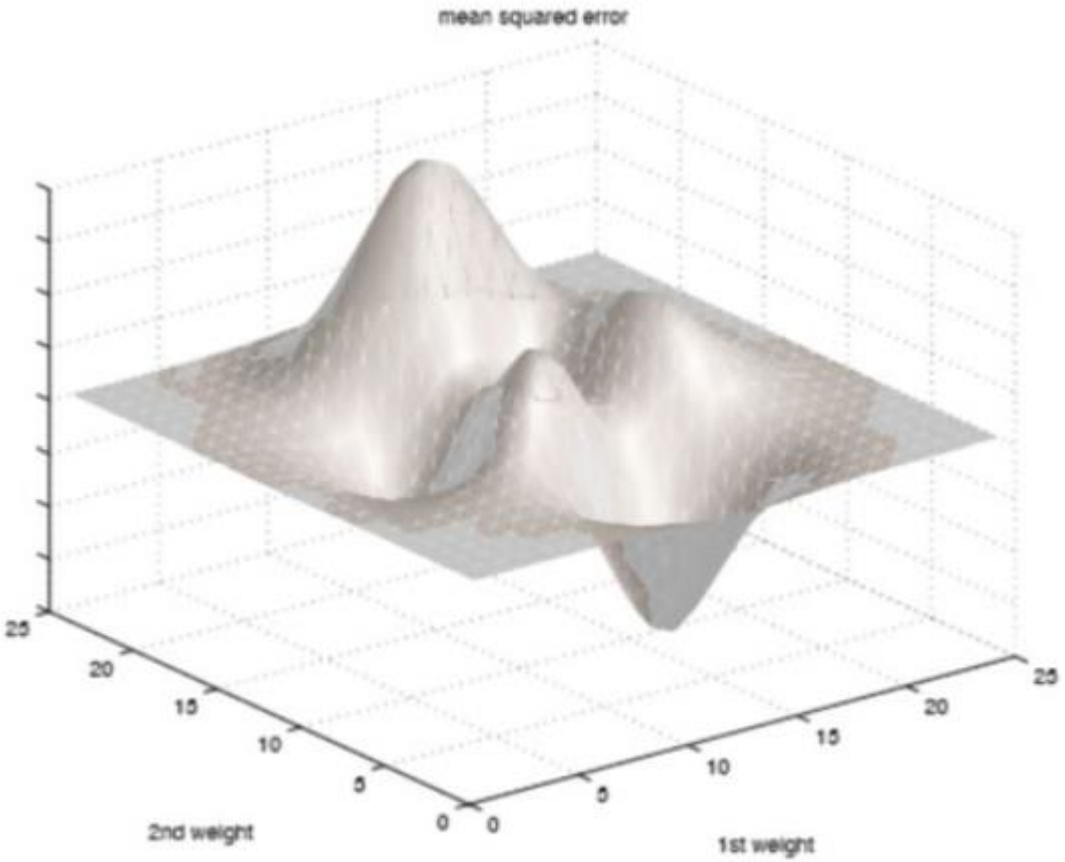
- Prosedür, doğrusal sınıflandırıcılar tarafından kullanılana benzerdir.
- Başlangıçta ağırlıklara, tipik olarak [−0.1, 0.1] aralığından rasgele sayılar atanır.
- Eğitim örnekleri daha sonra birer birer sunulur ve her biri çıktı katmanına iletilir. Ağın çıktısı ile örneğin hedef vektörü arasındaki tutarsızlık, ağırlıkların ne kadar değiştirileceğini belirler.
- Ağırlık değişikliğinden sonra, bir sonraki örnek sunulur. Son eğitim örneğine ulaşıldığında bir **epoch** tamamlanmıştır. Çok katmanlı algılayıcılarda, başarılı bir eğitim için gereken epoch sayısı, doğrusal sınıflandırıcılardakinden çok daha fazladır: binlerce, on binlerce, hatta daha fazla olabilir.

Hatanın Geri Yayılımı (Back Propagation)

Yerel Minimum ve Global Minimum

- Şekil, sorunun doğasını anlamamıza yardımcı olur. Dikey eksen, yatay eksenler boyunca çizilen ağın ağırlıklarının bir fonksiyonu olan ortalama kare hatasını temsil eder.

- Kolaylık sağlamak için, sadece iki ağırlık varmış gibi davranıyoruz – bu gerçekçi değil ama ikiden fazlasını kullanamayız çünkü çok boyutlu uzayları görselleştiremiyoruz.



Hatanın Geri Yayılımı (Back Propagation)

- Buradaki espri, hata fonksiyonunun, "vadilerinin" fonksiyonun yerel minimumunu temsil ettiği bir tür "manzara" olarak tasavvur edilebilmesidir.

- En derini (en küçük hata) küresel minimumdur ve eğitim prosedürünün ideal olarak ulaşması gereken şey budur; daha doğrusu eğitim, küresel minimuma karşılık gelen ağırlık kümesini bulmak ister.

Hatanın Geri Yayılımı (Back Propagation)

- Gradyan İniş / Dereceli Düşme (Gradient Descent)
- Herhangi bir ağırlık çifti verilen eğitim örneği için tipik olarak eğimlerden birinde bir yerde somut bir konum tanımlar.
- Herhangi bir ağırlık değişikliği, yatay eksenler boyunca farklı koordinatlara ve dolayısıyla farklı bir hata fonksiyonu değerine neden olacaktır.
- Bu yeni konumun tam olarak nerede olduğu, eğimin "yukarı" veya "aşağı" olup olmadığı, ağırlıkların her birinin ne kadar ve hangi yönde değiştiğine bağlı olacaktır.
- Örneğin, hem w_1'i hem de w_2'yi 0,1 artırmak, ortalama kare hatasında yalnızca küçük bir azalmaya yol açarken, w_1'i 0,3 ve w_2'yi 0,1 artırmak onu önemli ölçüde azaltır.
- Aşağıda açıklanan teknik, hata fonksiyonunda en dik düşüşe yol açan ağırlık değişikliklerini arar. Matematik dilinde buna **gradyan iniş** denir.

Hatanın Geri Yayılımı (Back Propagation)

- Sezgisel olarak, bireysel nöronların genel MSE'ye katkıları farklıdır: bazıları oyunu diğerlerinden daha fazla bozar.
- Eğer durum buysa, o zaman "kötü" nöronlara götüren bağlantılar, daha az saldırgan olanlara götüren bağlantılardan daha önemli ağırlık uyarlamalarından geçmelidir; fazla zarar vermeyen nöronların fazla değişime ihtiyacı yoktur.
- Genel hata için her nöronun sorumluluğu oldukça kolay bir şekilde belirlenir. Somut formüller, hangi transfer fonksiyonunun kullanılmış olduğuna bağlıdır. Fonksiyon sigmoid ise sorumluluk aşağıdaki gibi hesaplanır:

Çıkış katman nöronları : $\delta_i^{(1)} = y_i(1 - y_i)(t_i - y_i)$

Hatanın Geri Yayılımı (Back Propagation)

Çıkış katman nöronları : $\delta_i^{(1)} = y_i(1 - y_i)(t_i - y_i)$

- $(t_i - y_i)$'nin ağın i. çıkışı ile ilgili hedef değeri arasındaki fark olduğuna dikkat edin. Bu fark $yi(1 - y_i)$ ile çarpılır. İkinci terim aslında sigmoid'in ilk türevidir ve minimumuna $y_1 = 0$ veya $y_i = 1$ olduğunda ulaşılır.

- Terim, $y_i = 0.5$ olduğunda maksimize edilir, bu durumda "görüş" tarafsız kabul edilir. $\delta_i^{(1)}$'in işaretinin yalnızca $(t_i - y_i)$'ye bağlı olduğuna dikkat edin

- Çünkü $y_i(1 - y_i)$ her zaman pozitiftir.

Hatanın Geri Yayılımı (Back Propagation)

- Gizli nöronların sorumlulukları, bir önceki adımda elde edilen çıkış nöronlarının sorumlulukları $\delta_i^{(1)} w_{ji}$'nin geriye yayılmasıyla hesaplanır.

- $\delta_i^{(1)} w_{ji}$ teriminin rolü budur. Her $\delta_i^{(1)}$'in (i. çıkış nöronunun sorumluluğu), i. çıkış nöronu ile j. gizli nöronu birbirine bağlayan bağlantının ağırlığı ile çarpıldığına dikkat edin.

- Ağırlıklı toplam, $h_j(1 - h_j)$ ile çarpılır; bu, y_i'nin yerinin h_j tarafından alınması dışında, önceki adımda kullanılanla esasen aynı terimdir.

Gizli katman nöronları: $\delta_j^{(2)} = h_j(1 - h_j) \sum_i \delta_i^{(1)} wji^{(1)}$

Hatanın Geri Yayılımı (Back Propagation)

- Artık tek tek nöronların sorumluluklarını bildiğimize göre, onlara giden bağlantıların ağırlıklarını güncellemeye hazırız.

- Benzer şekilde, algılayıcı öğrenme (perceptron learning) durumunda olduğu gibi, toplama kuralı kullanılır:

çıkış katman nöronları: $w_{ji}^{(1)} := w_{ji}^{(1)} + \eta\delta_i^{(1)}h_j$
gizli katman nöronları: $w_{kj}^{(2)} := w_{kj}^{(2)} + \eta\delta_j^{(2)}x_k$

Hatanın Geri Yayılımı (Back Propagation)

- Ağırlık düzeltmesinin boyutu $\eta\delta_i^{(1)}hj$ veya $\eta\delta_j^{(2)}x_k$ ile belirlenir.
- İki gözlem yapılabilir. İlk olarak, nöronların sorumlulukları $\delta_i^{(1)}$ ve $\delta_j^{(2)}$, teorik olarak $\eta \in (0, 1)$ birim aralığından olan öğrenme oranı olan η ile çarpılır, ancak genellikle çok daha küçük değerler kullanılır, tipik olarak 0.1'den azdır.
- İkinci olarak, sonuçlar sırasıyla $h_j \in (0, 1)$ ve $x_k \in [-1, 1]$ ile çarpılır. Düzeltme bu nedenle küçüktür. Gerçek etkisi görecelidir.
- Eklenen terimin değeri, örneğin 0,02 ise, $w_{ij}^{(1)} = 0{,}01$ gibi daha küçük ağırlıklar, $w_{ij}^{(1)} = 1{,}8$ gibi daha büyük ağırlıklardan daha fazla etkilenecektir.

Hatanın Geri Yayılımı (Back Propagation)

Tek gizli katmanlı MLP'de hatanın geri yayılımı:

1. Present example x at the network's input and forward-propagate it.
2. Let $y = (y_1, \ldots y_m)$ be the output vector, and let $t(x) = (t_1, \ldots t_m)$ be the target vector.
3. For each output neuron, calculate its responsibility, $\delta_i^{(1)}$, for the network's error:

$$\delta_i^{(1)} = y_i(1 - y_i)(t_i - y_i)$$

4. For each hidden neuron, calculate its responsibility, $\delta_j^{(2)}$, for the network's error. While doing so, use the responsibilities, $\delta_i^{(1)}$, of the output neurons as obtained in the previous step.

$$\delta_j^{(2)} = h_j\,(1 - h_j)\sum_i \delta_i^{(1)} w_{ji}(1)$$

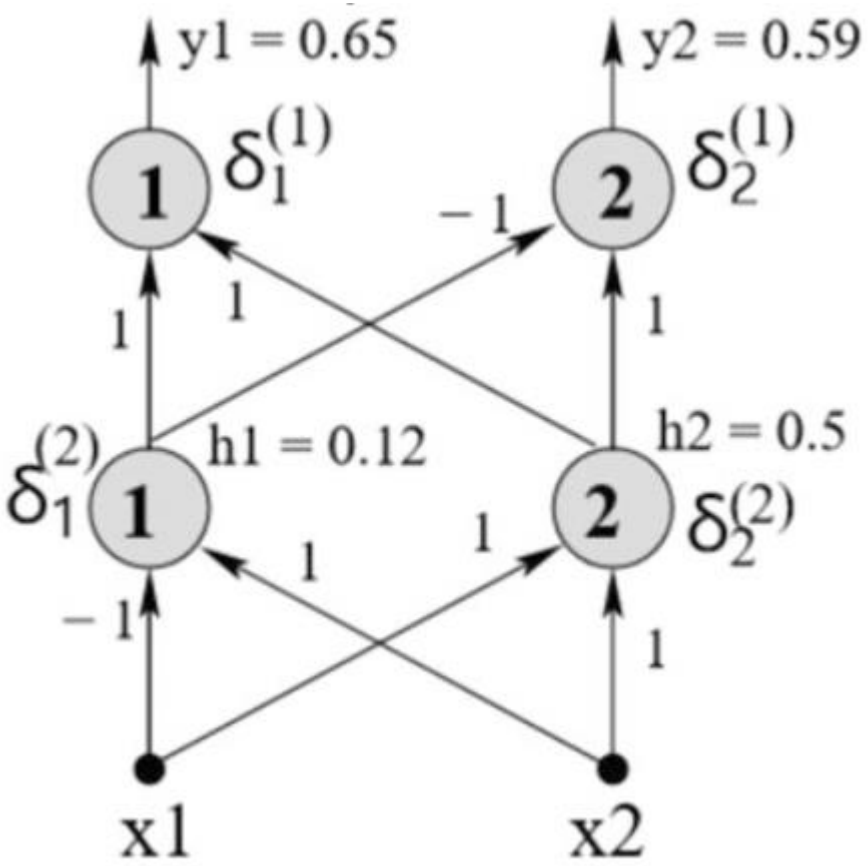
5. Update the weights using the following formulas, where η is learning rate:

output layer: $w_{ji}^{(1)} := w_{ji}^{(1)} + \eta\delta_i^{(1)} h_j$; h_j : the output of the j-th hidden neuron

hidden layer: $w_{kj}^{(2)} := w_{kj}^{(2)} + \eta\delta_j^{(2)} x_k$; x_k: the value of the k-th attribute

6. Unless a termination criterion has been satisfied, return to step 1.

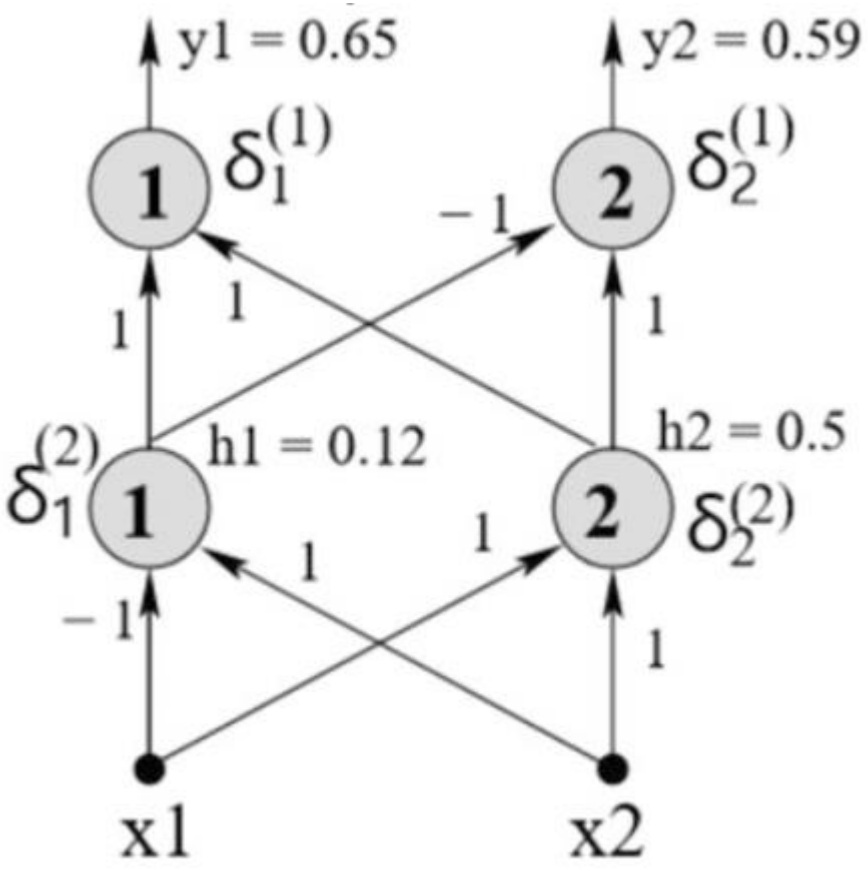
Hatanın Geri Yayılımı (Back Propagation)



- Yandaki sinir ağında transfer fonksiyonu f(Σ) = $\frac{1}{1+e^{-\Sigma}}$ olsun. Hatanın geriye yayılımını kullanarak (η=0.1 ile), aşağıdaki örneğin sunumundan sonra ağırlıkların nasıl değiştirildiğini gösterelim: [x, t(x)] = [(1, –1), (1, 0)]. Gerekli hesaplamaları azaltmak için bias x_0 göz ardı edilir.

$$h_1 = \frac{1}{1+e^{-(-2)}} = 0.12$$
$$h_2 = \frac{1}{1+e^{0}} = 0.5$$
$$y_1 = \frac{1}{1+e^{-(0.12+0.5)}} = 0.65$$
$$y_2 = \frac{1}{1+e^{-(-0.12+0.5)}} = 0.59$$

Hatanın Geri Yayılımı (Back Propagation)



$$\delta_1^{(1)} = y_1(1-y_1)(t_1-y_1) = 0.65(1-0.65)(1-0.65) = 0.0796$$
$$\delta_2^{(1)} = y_2(1-y_2)(t_2-y_2) = 0.59(1-0.59)(0-0.59) = -0.1427$$

$$\Delta_1 = \delta_1^{(1)} w_{11}^{(1)} + \delta_2^{(1)} w_{12}^{(1)} = 0.0796 \times 1 + (-0.1427) \times (-1) = 0.2223$$
$$\Delta_2 = \delta_1^{(1)} w_{21}^{(1)} + \delta_2^{(1)} w_{22}^{(1)} = 0.0796 \times 1 + (-0.1427) \times 1 = -0.0631$$
$$\delta_1^{(2)} = h_1(1-h_1)\Delta_1 = 0.12(1-0.12) \times 0.2223 = 0.0235$$
$$\delta_2^{(2)} = h_2(1-h_2)\Delta_2 = 0.5(1-0.5) \times (-0.0631) = -0.0158$$

$$w_{11}^{(1)} = w_{11}^{(1)} + \eta\delta_1^{(1)} h_1 = 1 + 0.1 \times 0.0796 \times 0.12 = 1.00096$$
$$w_{21}^{(1)} = w_{21}^{(1)} + \eta\delta_1^{(1)} h_2 = 1 + 0.1 \times 0.0796 \times 0.5 = 1.00398$$
$$w_{12}^{(1)} = w_{12}^{(1)} + \eta\delta_2^{(1)} h_1 = -1 + 0.1 \times (-0.1427) \times 0.12 = -1.0017$$
$$w_{22}^{(1)} = w_{22}^{(1)} + \eta\delta_2^{(1)} h_2 = 1 + 0.1 \times (-0.1427) \times 0.5 = 0.9929$$
$$w_{11}^{(2)} = w_{11}^{(2)} + \eta\delta_1^{(2)} x_1 = -1 + 0.1 \times 0.0235 \times 1 = -0.9977$$
$$w_{21}^{(2)} = w_{21}^{(2)} + \eta\delta_1^{(2)} x_2 = 1 + 0.1 \times 0.0235 \times (-1) = 0.9977$$
$$w_{12}^{(2)} = w_{12}^{(2)} + \eta\delta_2^{(2)} x_1 = 1 + 0.1 \times (-0.0158) \times 1 = 0.9984$$
$$w_{22}^{(2)} = w_{22}^{(2)} + \eta\delta_2^{(2)} x_2 = 1 + 0.1 \times (-0.0158) \times (-1) = 1.0016$$

Pratikte MLP’ler

- Çok katmanlı algılayıcıları eğitmek bilimden çok sanattır. Mühendis, deneyime ve MLP'nin çeşitli yönlerine ilişkin derin anlayışa güvenir. Akılda tutulması gereken daha önemli özelliklerden bazılarını kısaca gözden geçirelim.

Hesaplama Maliyetleri

- Hatanın geriye yayılımı hesaplama açısından pahalıdır. Bir örnek sunumu üzerine, her bir nöronun sorumluluğu hesaplanmalı ve ağırlıklar buna göre değiştirilmelidir. Bu, her eğitim örneği için, genellikle birçok epoch için tekrarlanır.

Pratikte MLP'ler

- Tüm bunların gerçek maliyeti hakkında bir fikir edinmek için, 1000 öznitelikle tanımlanan örnekleri sınıflandıracak bir ağ düşünün. 100 tane gizli nöron varsa bu alt katmandaki ağırlık sayısı $1000 \times 100 = 10^5$ olur.

- Bu her eğitim örneğinden sonra kaç tane ağırlık değiştirilmesi gerektiğidir. Sınıf sayısı az olduğu sürece üst katman ağırlıkları ihmal edilebilir. Örneğin, üç sınıflı bir durumda üst katman ağırlıklarının sayısı $100 \times 3 = 300$ olup, alt katmandaki 10^5 ağırlıktan çok daha azdır. Eğitim setinin 10^4 örnekten oluştuğunu ve eğitimin 10^4 epoch boyunca çalışacağını varsayalım.

- O halde ağırlık güncellemelerinin sayısı $10^5 \times 10^4 \times 10^4 = 10^{13}$'tür. Bu çok fazla gibi görünebilir ancak sıra dışı bir durum değildir; birçok uygulama çok daha zahmetlidir. Daha verimli eğitim yöntemleri geliştirilmiştir, ancak bunların ayrıntılı açıklaması, bu dersin kapsamı dışındadır.

Pratikte MLP'ler

Ayrık Niteliklerin Tek Sıcak Temsili (One-Hot Encoding)

- Çok katmanlı algılayıcılar, üstesinden gelinmesi kolay olan sürekli değerli nitelikler bekler.
- Boole özniteliklerinin değerleri, doğru için 1 ve yanlış için 0 ile değiştirilebilir.
- Çok değerli ayrık öznitelikler söz konusu olduğunda, tek sıcak temsil oldukça popülerdir. Fikir, n-değerli özniteliği n tane ikili öznitelikle değiştirmektir; bunlardan yalnızca biri 1'e ("sıcak"), diğerleri ise 0'a ayarlanır.
- Örnek olarak, mevsimin dört değeri vardır: bahar, yaz, sonbahar ve kış. Tek seferlik temsil, bu özelliği her sezon için bir tane olmak üzere dört ikili özellik ile değiştirir. Bu 4'lü demette, örneğin, ilkbahar (1, 0, 0, 0) olarak temsil edilir ve sonbahar (0, 0, 1, 0) olarak temsil edilir.

Pratikte MLP'ler

Toplu İşleme (Batch Processing)

- Ağırlık ayarlamalarının her eğitim örneği sunumundan sonra yapılması gerekmez.
- Alternatif bir yöntem, ağırlık değişikliklerini her örnek sunumda önerildiği şekilde toplar, ancak gerçek ağırlık değişimini yalnızca m eğitim örneğinin sunumundan sonra, eğitilebilir her parametreye birikmiş önerilerin toplamını ekleyerek gerçekleştirir.
- Toplu eğitim yalnızca hesaplama açısından daha ucuz değil, aynı zamanda uygulanması daha kolaydır çünkü modern programlama dilleri genellikle matris işlemlerini destekler.

Tekrar Ziyaret Edilen Hedef Değerler

- Basit olması için şimdiye kadar her hedef değerin 1 veya 0 olduğunu varsaydık. Bu en iyi seçim olmayabilir. Birincisi, bu değerlere bir nöronun çıktısı olan y_i ile asla ulaşılamaz çünkü sigmoid fonksiyonunun aralığı açık aralıktır (0, 1).

- Ayrıca, bu iki ucun çevresindeki ağırlık değişimleri çok küçüktür çünkü çıkış nöronunun sorumluluğunun hesaplanması, $\delta_i^{(1)} = y_i(1-y_i)(t_i - y_i)$, her y_i 0 veya 1' yaklaştığında sıfıra çok yakın bir değer verir.

Tekrar Ziyaret Edilen Hedef Değerler

- Son olarak, sınıflandırıcının çıktı nöronu en yüksek değeri döndüren sınıfı seçtiğini biliyoruz. Bireysel nöronun çıktı kesinliği bu nedenle o kadar önemli değildir; önemli olan diğer çıktılarla karşılaştırılmasıdır.

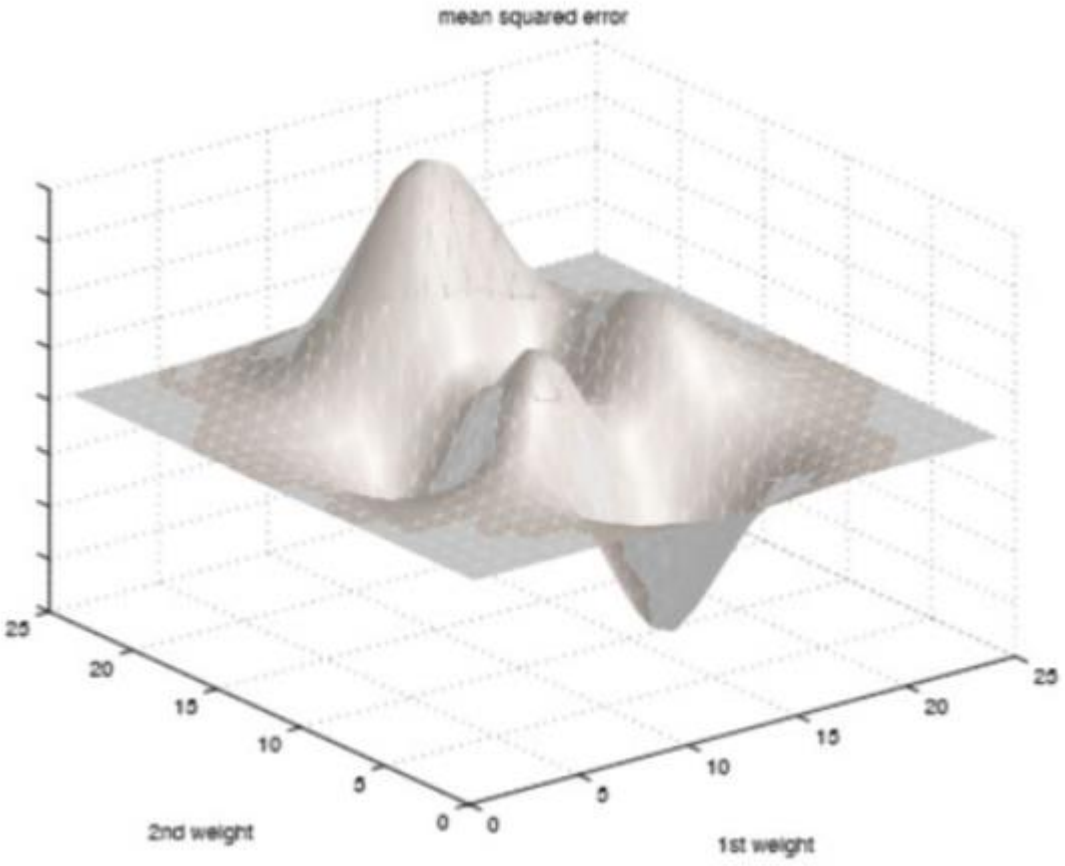
- İleri yayılım y = (0.9, 0.1, 0.2) ile sonuçlanırsa, örnek birinci sınıfla (y_i = 0.9 tarafından desteklenen) etiketlenmek zorundadır ve bu karar küçük ağırlık değişikliklerinden etkilenmeyecektir.

Pratikte MLP'ler

Yerel Minimum (Local Minima)

- Şekil, MLP eğitimine kademeli iniş (gradient descent) yaklaşımının ana dezavantajını göstermektedir.

- Ağırlıklar en dik eğimi takip edecek şekilde değiştirilir, ancak bir yerel minimumun tabanına ulaşıldığında, gidecek başka yer yoktur – ki bu gariptir: sonuçta, nihai hedef küresel minimuma ulaşmaktır.



Pratikte MLP'ler

Yerel Minimum (Local Minima)

- Çözüm için iki şeye ihtiyaç vardır: birincisi, yerel minimumu küresel minimumdan ayırt edecek bir mekanizma; ikincisi, yerel minimuma düşmekten kurtulmak için bir yöntem.
- Eğitim sırasında yerel minimumları belirlemenin bir yolu, ortalama karesel hatayı (MSE) takip etmek ve her dönemin sonunda tüm eğitim seti üzerinden özetlemektir. Normal şartlar altında, eğitim bir epoch'tan diğerine ilerledikçe bu miktar küçülme eğilimindedir.
- Neredeyse hiç hata azalmasının gözlemlenmediği bir platoya ulaştığında, öğrenme sürecinin yerel bir minimuma hapsolduğundan şüphelenilir. Bu zorluğun üstesinden gelme teknikleri genellikle uyarlanabilir öğrenme oranlarına ve yeni gizli nöronlar eklemeye dayanır. Genel olarak konuşursak, problem çok sayıda gizli nörona sahip ağlarda daha az kritiktir. Ayrıca, tüm ağırlıklar, örneğin (-0.01, 0.01) aralığından çok küçükse, yerel minimumlar daha sığ ve daha seyrek olma eğilimindedir.

Zamana Bağlı Öğrenme Oranı (Time Dependent Learning Rate)

- Hatanın geriye yayılmasını açıklarken, sabit bir öğrenme oranı η varsaydık. Gerçekçi uygulamalarda bu nadiren olur.
- Oldukça sık olarak, eğitim daha yüksek bir η değeriyle başlar ve zamanla kademeli olarak azalır.
- Başlangıçta, daha büyük ağırlık değişiklikleri dönem sayısını azaltır ve hatta MLP'nin bazı yerel minimumların "üzerinden atlamasına" yardımcı olabilir. Ancak daha sonra, bu büyük η küresel minimumun aşılmasına neden olabilir ve bu nedenle değerinin düşürülmesi gerekir. Öğrenme oranını zamanın bir fonksiyonu olarak ifade edersek, η(t), burada t bize kaç eğitim döneminin (epoch) tamamlandığını söyler, o zaman aşağıdaki negatif-üstel formül öğrenme oranını kademeli olarak azaltacaktır (α, negatifin eğimidir) üstel ve η(0) öğrenme oranının başlangıç değeridir):

$$\eta(t) = \eta(0)e^{-\alpha t}$$

Uyarlanabilir Öğrenme Hızı (Adaptive Learning Rate)

- Belki de bazı gelişmiş ağırlık değiştirme formüllerinin "mevcut eğilimleri" yansıtma yeteneğine sahip olduğuna dikkat edilmelidir.

- Örneğin, popüler bir çözüm, bir "momentum" uygulamaya çalışmaktan oluşur: son iki ağırlık değişikliği aynı yöndeyse (hem pozitif hem de negatif), ağırlık değiştirme adımını artırmak mantıklıdır; tersine, pozitif bir değişikliğin ardından negatif bir değişiklik geldiyse (tersi), ağırlık değiştirme adımı, aşırıya kaçmayı önlemek için azaltılmalıdır.

Pratikte MLP'ler

Aşırı Eğitim (Overtraining)

- Yeterince büyük çok katmanlı algılayıcılar, herhangi bir karar yüzeyini modelleme yeteneğine sahiptir ve bu, eğitim çok uzun sürerse, eğitim setini fazla uydurmaya yatkın hale getirir.
- Aşırı uydurmanın (overfitting) tipik olarak, test örneklerinde hayal kırıklığı yaratan performansla gürültülü eğitim örneklerinin mükemmel şekilde sınıflandırılması anlamına geldiğini hatırlayacaksınız.
- Bu fenomen genellikle ağın aşırı eğitimi olarak adlandırılır. Çok fazla eğitilebilir parametreye sahip olmayan küçük MLP'lerde sorun o kadar acı verici değildir. Ancak gizli nöronların sayısı arttıkça, ağ esneklik kazanır ve aşırı eğitim/fazla uyum endişe nedeni olabilir. Ancak bir sonraki bölümde öğreneceğimiz gibi bu her zaman küçük ağları tercih etmemiz gerektiği anlamına gelmiyor. Küçük ağların kendi sorunları vardır.

Pratikte MLP'ler

Doğrulama Seti (Validation Set)

- Aşırı eğitimi tespit etmenin basit bir yöntemi var. Eğitim seti yeterince büyükse, doğrulama seti için %10-20 kadar örnek bırakabiliriz.
- Doğrulama seti hiçbir zaman hatanın geri yayılımı için kullanılmaz; bağımsız veriler üzerindeki performansın nasıl geliştiğini gözlemlememize yardımcı olmak için kullanılır.
- Her dönemden (epoch) sonra, eğitime ara verilir ve geçerli MLP sürümü doğrulama setinde test edilir. Başlangıçta, doğrulama verilerindeki ortalama karesel hataların toplamı düşme eğilimindedir, ancak yalnızca belirli bir âna kadar; daha sonra tekrar büyümeye başlar ve mühendisi eğitimin artık verileri gereğinden fazla uydurma eğiliminde olduğu konusunda uyarır.

Başka bir Aktivasyon Fonksiyonu: tanh

- Bütünlük adına, klasik sinir ağlarında bazen sigmoid yerine başka bir aktivasyon fonksiyonunun kullanıldığını belirtmeliyiz: hiperbolik tanjant, tanh.

- S sigmoid fonksiyon olsun. tanh şu şekilde tanımlanır:

$$\tanh = 2S - 1$$

- Bununla birlikte, nöral aktivasyon fonksiyonu tanh ise, hatanın geri yayılımı için farklı formüller kullanılmalıdır.

Büyük Ağ mı Küçük Ağ mı?

- Kaç tane gizli nöron kullanılacak?
- Yalnızca bir veya iki tane varsa, ağ esneklikten yoksun olacaktır: karmaşık bir karar yüzeyi geliştiremeyecek ve eğitimi yerel minimumlar tarafından engellenecektir.
- Diğer uçta, binlerce gizli nöron kullanmak, çok sayıda nöronu eğitme ihtiyacı nedeniyle çok yüksek hesaplama maliyetleri anlamına gelir. Ayrıca, çok büyük ağlar aşırı derecede esnektir ve bu nedenle eğitim setini aşmaya eğilimlidir.
- İki aşırı uç zararlı olduğundan, bir uzlaşmaya ihtiyaç vardır.

Büyük Ağ mı Küçük Ağ mı?

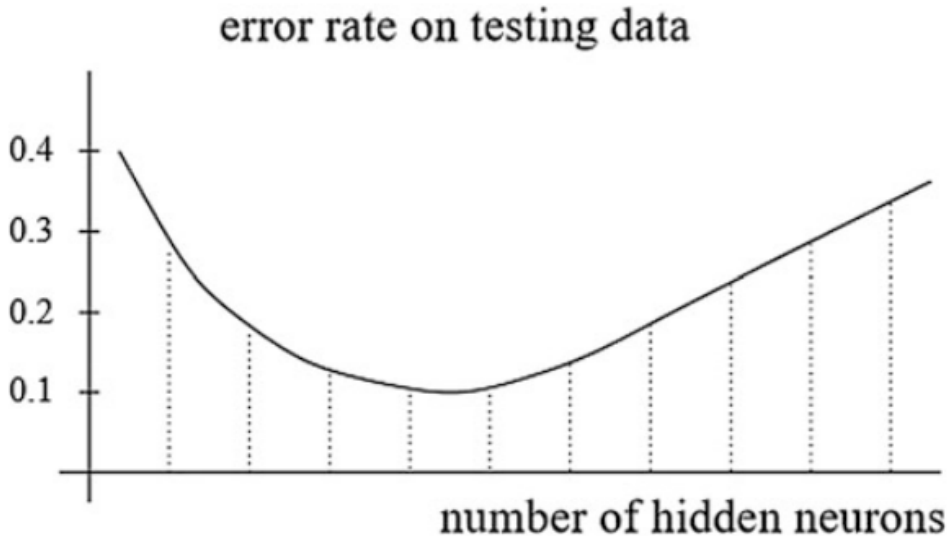
Performansa Karşı Boyut

- Aşağıdaki deneyi yaptığınızı varsayalım.
- Mevcut önceden sınıflandırılmış örnekler seti, biri eğitim, diğeri test için olmak üzere iki bölüme ayrılmıştır.
- Eğitim, her biri farklı sayıda gizli nörona sahip birkaç sinir ağı tarafından gerçekleştirilir. Ağlar, eğitim seti hata oranında hiçbir azalma gözlemlenmeyene kadar eğitilir. Bundan sonra, test verilerindeki hata oranı ölçülür.

Büyük Ağ mı Küçük Ağ mı?

Optimum Nöron Sayısı

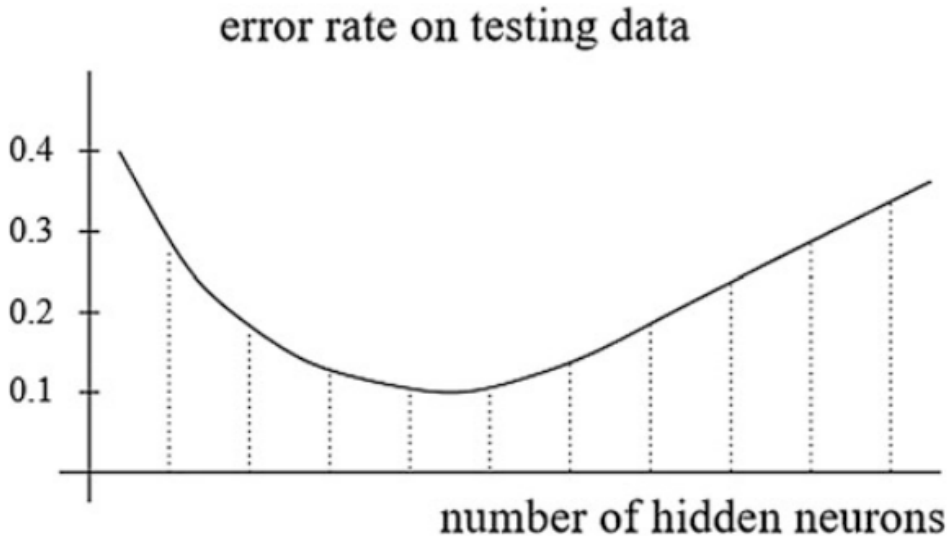
- Önceki paragraftaki deneyin sonuçları tipik olarak şekildeki gibi olacaktır.
- Burada yatay eksen, gizli nöronların sayısını temsil eder ve dikey eksen, bağımsız bir test setinde ölçülen hata oranını temsil eder.
- Tipik olarak, çok küçük ağlar söz konusu olduğunda hata oranı yüksek olacaktır çünkü küçük ağlar yeterli esnekliğe sahip değildir ve ayrıca yerel minimumların neden olduğu tehlikelerden muzdariptir.
- Gizli nöronların sayısını arttırırsak bu zayıflıklar hafifletilir. Grafikte gösterildiği gibi, bu daha büyük ağlar daha düşük hata oranları sergiler.



Büyük Ağ mı Küçük Ağ mı?

Optimum Nöron Sayısı

- Bununla birlikte, çok büyük ağlar, verileri aşırı zorlama ve dolayısıyla verileri aşırı yükleme eğilimindedir.
- Belirli bir noktadan sonra, test kümesi hatası yeniden büyümeye başlar (grafikteki sağ kısma bakın). Bu hata eğrisinin kesin şekli, eğitim verilerinin karmaşıklığına bağlıdır. Basit domain'lerde, ağ 3-5'ten fazla gizli nöron içermediğinde hata oranı en aza indirilir.
- Zor domain'lerde, minimuma yalnızca yüzlerce gizli nöron kullanıldığında ulaşılır. Eğer eğitim örnekleri tamamen gürültüsüz ise fazla uydurma daha az sorun olur ve eğrinin sağ kuyruğu yalnızca orta derecede büyüyebilir.



Büyük Ağ mı Küçük Ağ mı?

Uygun Büyüklüğü Aramak

- Yukarıda açıklanan senaryo, pratik uygulamalar için çok pahalıdır. Ne de olsa, sadece birkaç nörona mı, yoksa düzinelerce mi yoksa yüzlerce mi ihtiyacımız olacağı konusunda hiçbir fikrimiz yok ve boyutu belirlemeden önce hesaplama açısından yoğun eğitimi birçok kez tekrarlamamız gerekebilir.

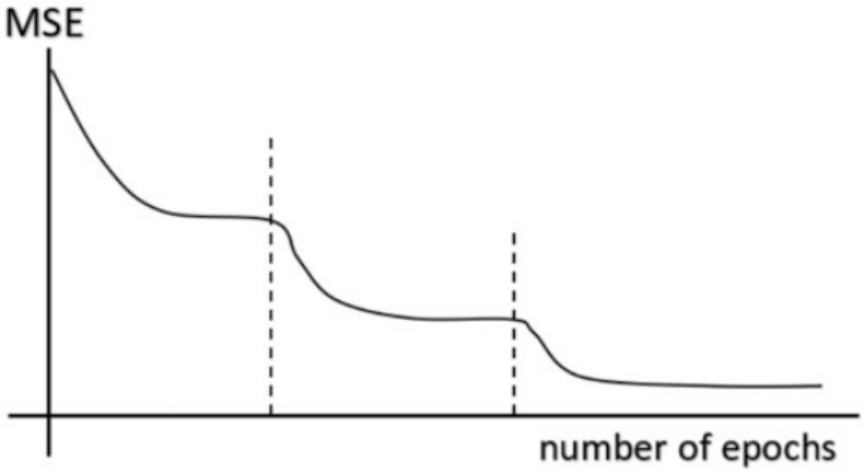
- Bu nedenle, doğru boyutu daha verimli bir şekilde bulabilen bazı teknikler bilmek iyi olacaktır. Böyle bir teknik, aşağıdaki sözde kodla özetlenmiştir:

1. At the beginning, use only a few hidden neurons.
2. Train the network until the mean squared error no longer seems to improve.
3. At this moment, add a few neurons to the hidden layer, each with randomly initialized weights, and resume training.
4. Repeat the previous two steps until a termination criterion has been satisfied; for instance, when the new addition does not result in a significant error reduction, or when the hidden layer exceeds a user-set maximum size.

Büyük Ağ mı Küçük Ağ mı?

Uygun Büyüklüğü Aramak

- Fikir, yalnızca birkaç gizli nöronu olan çok küçük bir ağla başlamaktır. Her aşamadan sonra, öğrenme algoritması, eğitim setinde gözlemlenen ortalama karesel hataların toplamını kontrol eder.
- Artan dönem sayısıyla birlikte hataların toplamının azalması muhtemeldir – ancak yalnızca belirli bir noktaya kadar. Buna ulaşıldığında, ağın performansı, ya yetersiz esnekliği nedeniyle ya da yerel bir minimuma takılıp kalması nedeniyle artık iyileşmez.
- Bu gözlemlendiğinde, rastgele başlatılan ağırlıklara sahip birkaç nöron daha eklenir ve eğitim kaldığı yerden devam eder. Genellikle, eklenen nöronlar daha fazla hata azalmasına yol açar. Şekilde MSE iki durumda düzelir; her ikisinde de yeni gizli nöronlar eklemek gerekli yeni esnekliği sağladı.



Büyük Ağ mı Küçük Ağ mı?

Birden Fazla Gizli Katmanlı Ağlar

- Şimdiye kadar sadece tek bir gizli katman ele alınmıştır. Bununla birlikte, pratik deneyimler, hesaplama maliyetleri ve sınıflandırma performansı söz konusu olduğunda, bazen iki veya daha fazla gizli katmana sahip sinir ağlarında daha iyi sonuçlara ulaşıldığını göstermektedir.

- Birden fazla gizli katmanlı ağlarda ileriye ve geriye doğru yayılma ilkeleri neredeyse hiç değişmeden kalır.

Büyük Ağ mı Küçük Ağ mı?

Kaybolan Gradyan (Vanishing Gradient)

- Sigmoid aktarım işlevine sahip MLP'ler söz konusu olduğunda, iki veya üçten fazla katman kullanmanın ağların kullanışlılığına neredeyse hiç katkıda bulunmadığı deneyimlenmiştir. Bunun nedeni, kaybolan gradyan sorunu olarak bilinir.

- Sigmoid transfer fonksiyonunda çıkış hata değerleri ($t_j - y_j$) çok küçük olabilir ve aynısı alt katmanlarda da olur. Ayrıca, $h_j^{(k)}(1-h_j^{(k)})$ çarpanı da küçüktür. Sonuç olarak, ağırlık değiştirme terimleri $w_{ji}^{(k)}$, k üst simgesinin değeri arttıkça çok küçük, neredeyse algılanamaz hale gelebilir.

- Örneğin üçten fazla gizli katman kullanıldığında, öğrenme süreci hesaplama açısından verimsiz hale gelir.

Büyük Ağ mı Küçük Ağ mı?

Birden Çok Gizli Katmanla İlgili Sonuç

- Ek gizli katmanların avantajı, sundukları ekstra esnekliktir. Dezavantajı, kaybolan gradyan fenomeninin neden olduğu öğrenme sürecinin verimsizliğidir.

- Ayrıca, ek katmanlar genellikle eğitilebilir parametrelerin sayısını artırır ve ne kadar çok eğitilebilir parametreye sahip olursak, aşırı eğitim tehlikesini azaltmak için o kadar çok eğitim örneğine ihtiyaç vardır.

- Derin öğrenme durumunda bazen birçok gizli katman kullanılır. Bu, sadece günümüz bilgisayarlarının çok daha yüksek gücü ile değil, aynı zamanda daha uygun transfer fonksiyonlarını kullanma fikri ile mümkün olmaktadır.

MLP'lere Modern Yaklaşımlar

- Son on yılda, yapay sinir ağları alanı, derin öğrenme olarak bilinen teknolojiyle sonuçlanan dramatik bir evrim geçirdi.
- En önemli değişikliklerden bazılarını özetleyelim.

MLP’lere Modern Yaklaşımlar

Şu Anda Popüler Aktivasyon Fonksiyonları

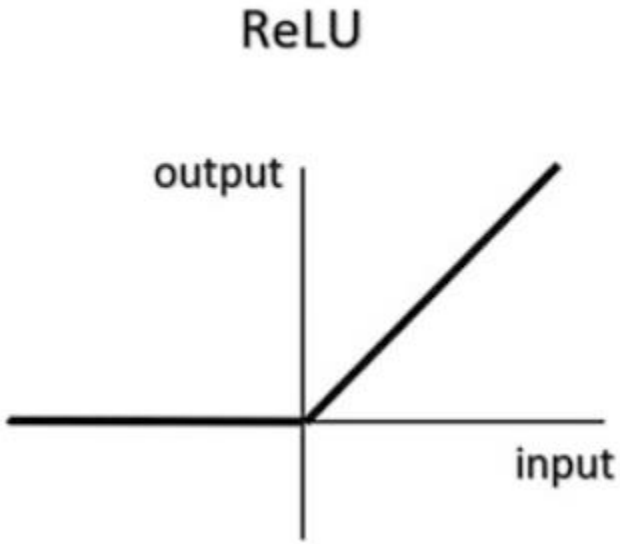
- Yapay sinir ağlarının temel ilkelerinden biri, sinir transfer fonksiyonlarının doğrusal olmaması gerektiğidir.

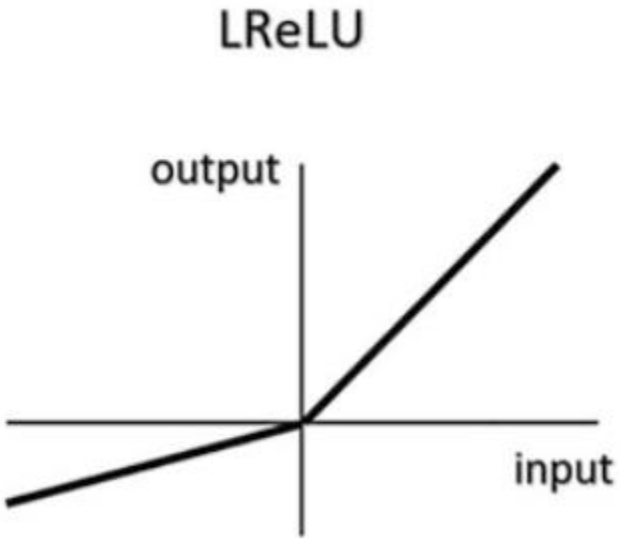
- İlk nesiller boyunca, sigmoid şeklinin burada özellikle yararlı olduğundan neredeyse hiç kimse şüphe duymuyordu. Bilimsel topluluk, çok daha basit parçalı lineer fonksiyonlardaki lineer olmama durumlarının başka avantajlar sunarken aynı derecede iyi olduğunu ancak son zamanlarda fark etti.

- Sonraki slayttaki şekil şu anda popüler olan iki etkinleştirme işlevini göstermektedir. Solda Rectified Linear Unit'in kısaltması olan ReLU ve sağda Leaky Rectified Linear Unit'in kısaltması olan LReLU var. Her iki fonksiyon da doğrusal olmayan özellikler içerir ve her ikisi de parçalı olsa bile türevlenebilirdir. Daha da önemlisi, sigmoid'den farklı olarak hiçbiri üst sınır değildir.

MLP'lere Modern Yaklaşımlar

Şu Anda Popüler Aktivasyon Fonksiyonları





MLP’lere Modern Yaklaşımlar

ReLU ve LReLU'nun Avantajları

- Parçalı doğrusal fonksiyonların ayırt edilmesi kolaydır. Girdi alanının herhangi bir aralığındaki birinci türev bir sabittir, hatta bazen sıfırdır.
- Bu tür türevlerin hesaplama açısından daha ucuz ve geri yayılmanın daha kolay olduğu ortadadır. ReLU ve LReLU'nun sigmoid'in açık aralık (0, 1) tarafından hapsedilmesinden muzdarip olmaması, ağın yalnızca 1'i aşan değerler vermesine izin vermekle kalmaz, aynı zamanda yok olan gradyan sorununu da neredeyse ortadan kaldırır.
- Bu aktivasyon işlevlerine sahip ağların, klasik sigmoid tabanlı MLP'lerin ana sınırlamalarından etkilenmeden birçok gizli katmana sahip olabileceği anlamına gelir: **DERİN AĞLAR!**

MLP'lere Modern Yaklaşımlar

Soft-Max: Çıktıları Olasılıklara Çevirmek

- Bir başka yeni yenilik de, ağın çıkış sinyallerinin belirli bir şekilde sonradan işlenmesidir.
- Soft-max, çıktıları, toplamı 1 olan olasılıklar olarak yorumlamaya izin verir.
- Verilen sinir ağının N çıktısının vektörü (y_1,..., y_N) olsun.
- Softmax işlevi, aşağıdaki denklemle bireysel çıktıları yeniden hesaplar:

$$p_i = \frac{e^{y_i}}{\Sigma_j e^{y_j}}$$

MLP'lere Modern Yaklaşımlar

MSE Yerine Bilgi Kaybı (Information Loss)

- Çıktıların artık olasılıklar olarak yorumlanabilmesi, belirli bir örnekteki ağın hatasının ölçülmesine ilginç bir alternatif sunuyor.
- Ortalama kare hatası yerine genellikle kayıp fonksiyonu kullanılır.
- C_i sınıfına ait bir örneğin bir MLP'nin girişine sunulduğunu ve bu örneğin öznitelik değerlerinin ağın çıktısına iletildiğini varsayalım. Softmax'tan sonra, i-inci çıkışın (C_i sınıfına karşılık gelen) p_i olduğunu varsayalım.
- Karar Ağaçlarındaki formül hatırlanacak olursa, "örnek i-inci sınıfa aittir" mesajının ilettiği bilgi miktarının şu şekilde hesaplanabilir:

$$L = -\log_2 p_i$$

Sorular

- Ders Sonu.